



Chapter 9

The Linear Algebra of Similarity

“The worst form of inequality is to try to make unequal things equal.” – Aristotle

9.1 Introduction

A dot-product similarity matrix is an alternative way to represent a multidimensional data set. In other words, one can convert an $n \times d$ data matrix D into an $n \times n$ similarity matrix $S = DD^T$ (which contains n^2 pairwise dot products between points). One can use S instead of D for machine learning algorithms. The reason is that the similarity matrix contains almost the same information about the data as the original matrix. This equivalence is the genesis of a large class of methods in machine learning, referred to as *kernel methods*. This chapter builds the linear algebra framework required for understanding this important class of methods in machine learning. The real utility of such methods arises when the similarity matrix is chosen differently from the use of dot products (and the data matrix is sometimes not even available).

This chapter is organized as follows. The next section discusses how similarity matrices are alternative representations of data matrices. The efficient recovery of data matrices from similarity matrices is discussed in Section 9.3. The different types of linear algebra operations on similarity matrices are discussed in Section 9.4. The implementation of machine learning algorithms with similarity matrices is discussed in Section 9.5. The representer theorem is discussed in Section 9.6. The choice of similarity matrices that promote linear separation is discussed in Section 9.7. A summary is given in Section 9.8.

9.2 Equivalence of Data and Similarity Matrices

This section will establish the rough equivalence between data matrices and similarity matrices. Therefore, the next subsection will concretely show how one can convert a data matrix into a similarity matrix and vice versa.

9.2.1 From Data Matrix to Similarity Matrix and Back

Consider an $n \times d$ data matrix D , in which the i th row is denoted by $\overline{X}_i = [x_{i1}, x_{i2}, \dots, x_{id}]$, and it corresponds to the i th object in the data set. Then, a *symmetric* $n \times n$ similarity matrix $S = [s_{ij}]$ can be defined among the n objects as follows:

$$s_{ij} = \overline{X}_i \cdot \overline{X}_j = \sum_{k=1}^d x_{ik}x_{jk}$$

One can write the above similarity relationship in matrix form as well:

$$S = DD^T$$

How does one recover the original data set D from the similarity matrix? First, note that the recovery can never be unique. This is because dot products are invariant to rotation and reflection. Therefore, rotating a data set about the origin or reflecting it along any axis will result in the same similarity matrix. For example, consider a $d \times d$ matrix P with orthonormal columns, which is essentially a rotation/reflection matrix. Then, the rotated/reflected version of D is as follows:

$$D' = DP$$

Then, the similarity matrix S' using D' can be shown to be equal to S as follows:

$$S' = D'D'^T = (DP)(DP)^T = D \underbrace{(PP^T)}_I D^T = S$$

In other words, the similarity matrices using D and D' are the same.

It is noteworthy that both $(DP)(DP)^T$ and DD^T represent *symmetric factorizations* of the similarity matrix S . A symmetric factorization of an $n \times n$ matrix is a factorization of S into two $n \times k$ matrices of the form $S = UU^T$. For exact factorization, the value of k will be equal to the rank of the similarity matrix S . The i th row of U in any symmetric factorization UU^T of S yields a valid set of features of the i th data point.

The simplest way to perform symmetric factorization of a similarity matrix is with the use of *eigendecomposition*. First, note that if S was indeed created using dot products on the data matrix D , it is of the form DD^T and is therefore positive semidefinite (cf. Lemma 3.3.14 of Chapter 3). Therefore, one can diagonalize it with nonnegative eigenvalues of which at most $\min\{n, d\}$ are non-zero. To emphasize the nonnegativity of the eigenvalues, we will represent the diagonal matrix as Σ^2 :

$$S = Q\Sigma^2Q^T = \underbrace{(Q\Sigma)}_U (Q\Sigma)^T$$

Therefore, $Q\Sigma$ is the extracted representation from the similarity matrix S , and it will contain at most $\min\{n, d\}$ non-zero columns. Specifically, the i th row of $Q\Sigma$ contains the embedded representation of the i th data point (based on the ordering of rows/columns of the similarity matrix S). Note that the eigenvectors and eigenvalues of $DD^T = Q\Sigma^2Q^T$ are (respectively) the left singular vectors and squared singular values of D .

The eigendecomposition of the similarity matrix provides one of an infinite number of possible embeddings obtained from factorization of the similarity matrix, and it is one of the most compact ones in terms of the number of non-zero columns. The compactness can be improved further by dropping smaller eigenvectors. As another example, one can

extract a symmetric Cholesky factorization $S = LL^T$, and use the rows of matrix L as the engineered representations of the points (see Section 3.3.9), although a small positive value might be needed to be added to each diagonal entry of S to make it positive definite. Another example is the symmetric *square-root* matrix, which can also be extracted from the eigendecomposition as $S = Q\Sigma^2Q^T = (Q\Sigma Q^T)(Q\Sigma Q^T)^T = (\sqrt{S})^2$. Choosing any particular embedding among these will not affect the predictions of any machine learning algorithm that relies on dot products (or Euclidean distances), because they remain the same whether we use eigendecomposition, Cholesky factorization, or the square-root matrix.

Problem 9.2.1 (Alternative Embeddings Are Orthogonally Related) *Show that if the rank- k similarity matrix S of size $n \times n$ can be expressed as either $U_1U_1^T$ or $U_2U_2^T$ with $n \times k$ matrices, then (i) a full SVD of each of U_1 and U_2 can be constructed so that the left singular vectors and the singular values are the same in the two cases, and (ii) an orthogonal matrix P_{12} can be found so that $U_2 = U_1P_{12}$.*

9.2.2 When Is Data Recovery from a Similarity Matrix Useful?

The above discussion simply creates a dot-product similarity matrix from a data matrix and then recovers a rotated/reflected version of the data set from it. At face value, this does not seem like a useful exercise. However, the real usefulness of this type of data recovery approach arises when the similarity matrix is constructed from the data using a method *different* from dot products. In fact, the original data set might not even be a multidimensional data type (and dot products are not possible). Rather, it might represent a set of structural data objects, such as small graph objects (e.g., chemical compounds), time-series, or discrete sequences. The similarity matrix S might have been created using a domain-specific similarity function on these objects. In such cases, the matrix $Q\Sigma$ of the scaled eigenvectors of similarity matrix $S = Q\Sigma^2Q^T$ contains *engineered features* of the objects in its rows. *In fact, the most common approach for feature engineering of arbitrary object types is the extraction of eigenvectors from the similarity matrix of the objects.*

Why is it useful to create multidimensional embeddings? One reason is that it is hard to apply machine learning algorithms on many data types such as discrete sequences or graph-centric chemical compounds. However, extracting a multidimensional embedding of each object opens the door to the use of many machine learning algorithms like SVMs or logistic regression that work with multidimensional data.

At most d eigenvectors of the $n \times n$ similarity matrix are non-zero, when the similarity matrix was truthfully created using dot products on a multidimensional data of dimensionality d . However, similarity matrices are rarely created using dot products on multidimensional data (and there is no practical use of doing so). For arbitrary similarity matrices created using domain-specific similarity functions on different types of objects, it is possible for all n eigenvalues of the $n \times n$ similarity matrix to be non-zero. One can interpret this result from the point of view that any n points in an embedding (and the origin) lie on an n -dimensional plane passing through the origin, although they might define an even lower dimensional plane if they are linearly dependent. The rows of matrix U in the factorization $S = UU^T$ contain the coordinates with respect to an n -dimensional orthogonal basis of this plane. When the $n \times n$ similarity matrix is extracted from the use of dot products on a multidimensional data set of dimensionality $d \ll n$, the linear dependence among the n embedded vectors ensures that the dimensionality of the hyperplane defined by these n vectors is no larger than d . However, this is not the case if one uses similarity functions other than the dot product on the original data set, which results in feature engineering from the data set.

9.2.3 What Types of Similarity Matrices Are “Valid”?

Similarity matrices are alternative representations of data sets (ignoring rotation and reflection). Even when the original data is not multidimensional (e.g., graph objects), one can simply assume that the similarities represent dot products between (fictitious) multidimensional objects. However, this assumption of the existence of a fictitious embedding needs to meet an important mathematical test of validity. As a small example, diagonal entries of similarity matrices are *non-negative* squared norms of embedded objects. Therefore, a similarity matrix with a negative diagonal entry could not have possibly been created using dot products even on a fictitious embedding.

A multidimensional embedding can be extracted from a similarity matrix S , if and only if it can be expressed in the form UU^T . Any matrix S expressible in this form must be positive semidefinite (cf. Lemma 3.3.14 of Chapter 3). *In other words, a similarity matrix needs to be positive semidefinite in order for a valid embedding to exist.*

Unfortunately, if an $n \times n$ similarity matrix S is extracted using a domain-specific similarity function, there is no guarantee that it will be positive semidefinite. What can be done in such cases? It turns out that it is always possible to repair any similarity matrix (without significantly changing the interpretation of the similarities) so that a valid embedding exists. The idea is to add $\delta > 0$ to each diagonal entry of the similarity matrix S , where δ is the magnitude of the most negative eigenvalue of the matrix S . In such a case, it can be shown that the resulting matrix $S' = S + \delta I$ is positive semidefinite.

$$S' = S + \delta I = Q\Delta Q^T + \delta I = Q \underbrace{(\Delta + \delta I)}_{\geq 0} Q^T$$

In this case, the embedding can be extracted as $Q\sqrt{\Delta + \delta I}$. The modification of the similarity matrix is often not a significant one from the perspective of application-centric interpretability. By doing so, we are only translating the (less important) self-similarity values to make them sufficiently large, while keeping the pairwise similarity values unchanged. Intuitively, the self-similarities among points are always larger than those across different points (on the average), when working with dot products.

Problem 9.2.2 Let $\bar{X}$ and $\bar{Y}$ be two d -dimensional points. Show that the average of the two dot-product self-similarities $\bar{X} \cdot \bar{X}$ and $\bar{Y} \cdot \bar{Y}$ is at least as large as the pairwise dot-product similarity $\bar{X} \cdot \bar{Y}$.

Stated differently, the above problem implies that if we have a 2×2 symmetric similarity matrix in which the sum of diagonal entries is smaller than the sum of off-diagonal entries, the matrix would not be positive semidefinite.

Problem 9.2.3 Show that the sum of the entries in a similarity matrix S can be expressed as $\bar{y}^T S \bar{y}$ for appropriately chosen column vector $\bar{y}$. What can you infer about the sign of the sum of the values in a similarity matrix that is positive semidefinite?

Problem 9.2.4 Let $\bar{y}$ be an n -dimensional column vector. Show that the expression $\bar{y}^T S \bar{y}$ represents the squared norm of some vector in the multidimensional space containing the embedding induced by the $n \times n$ similarity matrix S .

It is noteworthy that even though increasing the diagonal entries of S by δ affects the embedding matrix $Q\sqrt{\Delta + \delta I}$, it does not affect the *normalized* embedding matrix Q . In fact, several forms of feature engineering (such as *spectral embeddings*) work with normalized

embeddings. Although the importance of positive semidefiniteness is often emphasized in the machine learning literature on kernel methods, the reality is that this requirement is a lot less important (from a practical perspective) than appears at first glance – the similarity matrix can always be repaired by increasing the self-similarity entries along the diagonal (which are semantically less significant anyway).

9.2.4 Symmetric Matrix Factorization as an Optimization Model

The extraction of embeddings from similarity matrices is a special case of symmetric matrix factorization. However, it is not necessary to perform exact factorization. When one is looking to generate a k -dimensional embedding for $k \ll n$, the $n \times k$ embedding U defines a representation in which $\|S - UU^T\|_F^2$ is minimized. Therefore, one can pose the problem of finding a k -dimensional embedding as an unconstrained matrix factorization problem for an $n \times k$ matrix of variables U :

$$\text{Minimize}_U J = \frac{1}{2} \|S - UU^T\|_F^2$$

The top- k (scaled) eigenvectors $Q_k \Sigma_k$ of this similarity matrix $S = Q_k \Sigma_k^2 Q_k^T$ represent one of the solutions to this optimization problem. A particular property of this solution is that the columns of $U = Q_k \Sigma_k$ are mutually orthogonal. However, other alternative solutions to the optimization problem are possible. To understand this point, we recommend the reader to work out the following exercise:

Problem 9.2.5 *It is known that $U = Q_k \Sigma_k$ is one of the optimal solutions to the optimization problem presented above. Here, Q_k is an $n \times k$ matrix containing the top- k eigenvectors of S in its columns, and Σ_k is a diagonal matrix containing the square-root of the corresponding eigenvalues in its diagonal entries. Show that any solution of the form $U' = Q_k \Sigma_k R_k$ is also a solution to this optimization problem. Here, R_k is any $k \times k$ orthogonal matrix. Discuss why the columns of U' need not necessarily be mutually orthogonal.*

A hint for solving the last part of the above problem is to compute the expression for $U'^T U'$ and show that it will typically not be diagonal other than in some very special cases. As discussed in Section 9.3, it is possible to discover these alternative embeddings efficiently by using methods such as column sampling and stochastic gradient descent.

9.2.5 Kernel Methods: The Machine Learning Terminology

The linear algebra operations on similarity matrices are commonly used in kernel methods for machine learning. Therefore, we connect these linear algebra concepts with the terminology used in machine learning. The multidimensional representation extracted from a similarity matrix (using eigendecomposition) is referred to as a *data-specific Mercer kernel map*. More generally, these representations are referred to as *kernel feature spaces* or simply *feature spaces*.

Definition 9.2.1 (Kernel Feature Spaces) *The multidimensional data space obtained by diagonalizing the positive semidefinite similarity matrix between objects is referred to as a data-specific kernel feature space, or simply a data-specific feature space.*

We would like to emphasize that the definition of a kernel feature space here is a “data-specific” version, where a similarity matrix of finite size is already given, which contains *samples* of similarity values. The specification of a similarity matrix of finite size ensures

that the feature space is bounded above by the size of the similarity matrix. However, it is also possible to (implicitly) specify a similarity matrix of infinite size, by defining similarities between each pair of objects from an infinite domain *as a function in closed form*. For example, consider a situation where one wants to engineer new features $\Phi(\bar{x})$ from multidimensional vectors $\bar{x} \in \mathcal{R}^d$. In such a case, one can define a similarity function between multidimensional objects $\bar{x}$ and $\bar{y}$ (different from the dot product) by the following simplified *Gaussian kernel* with unit variance:

$$K(\bar{x}, \bar{y}) = \Phi(\bar{x}) \cdot \Phi(\bar{y}) = \exp(-\|\bar{x} - \bar{y}\|^2/2)$$

By providing a closed-form expression, one has effectively defined a similarity matrix between all pairs of objects $\bar{x}$ and $\bar{y}$ in the infinite set $\mathcal{R}^d$. Since the dimensionality of the eigenvectors increases with similarity matrix size, it is possible¹ for the resulting eigenvectors to also be infinite dimensional. Such spaces of infinite-dimensional vectors are natural generalizations of finite-dimensional Euclidean spaces, and are referred to as *Hilbert spaces*. However, even in these abstract cases of infinite-dimensional representations, it is possible to represent a specific data set of finite size containing n objects in n -dimensional space—the key point is that an n -dimensional projection of this infinite-dimensional space always exists that contains all n objects (and the origin). After all, any set of n vectors defines an (at most) n -dimensional *subspace*. The eigendecomposition of the *sample matrix* of size $n \times n$ discovers precisely this *subspace*, which is the *data-specific* feature space. For most machine learning problems, only the data-specific feature space is needed. We emphasize this point below:

For a finite similarity matrix of size $n \times n$, one can always extract an (at most) n -dimensional engineered representation using eigendecomposition of the similarity matrix. This is true even when the dimensionality of the true feature space induced by a (closed-form) similarity function over an infinite domain of points is much larger.

The key point is that as long as we do not need to know the representations of points outside our finite data set of n points, one can restrict the dimensionality of representation to an n -dimensional subspace (and much lower in many cases). In this chapter, whenever we refer to kernel feature space, we refer to the data-specific feature space whose dimensionality is bounded above by the number of points. The eigendecomposition of the similarity matrix to extract features is also referred to as *kernel SVD*.

Definition 9.2.2 (Kernel SVD) *The embedding $Q\Sigma$ extracted by the eigendecomposition $S = Q\Sigma^2Q^T$ of an $n \times n$ positive semidefinite similarity matrix S is referred to as kernel SVD. The $n \times n$ matrix $Q\Sigma$ contains the embedding of the i th data point in its i th row for each $i \in \{1 \dots n\}$. When S already contains dot products between points from $\mathcal{R}^d$, the approach specializes to standard SVD.*

All kernel methods in machine learning *implicitly* transform the data using kernel SVD via a method referred to as the “kernel trick.” However, we will revisit some traditional applications of kernels like SVMs and show how to implement them using *explicit* eigendecomposition of the similarity matrix. Although this approach is unusual, it is instructive and has some advantages over the alternative that avoids this eigendecomposition.

¹For some closed-form functions like the dot product, only d components of the eigenvectors will be non-zero, whereas for others like the Gaussian kernel, the entire set of infinite components will be needed.

9.3 Efficient Data Recovery from Similarity Matrices

The most basic approach for embedding extraction is the materialization of an $n \times n$ similarity matrix S from the n objects by computing all pairwise similarities, and then extracting large eigenvectors of S . However, this can be difficult to achieve in practice. Imagine a situation in which one has a billion data objects (e.g., chemical compounds), and therefore $n = 10^9$. By modern standards, a data set containing a billion objects is not considered extraordinarily large. In such a case, the number of entries in the similarity matrix is 10^{18} , which can be difficult to even materialize explicitly. In many cases, a closed-form function might be available to compute the similarities between each pair of objects, although one does not want to be forced to compute this function 10^{18} times. For example, if 1 cycle of a computer were required to compute a single pairwise similarity, then a 10 GHz computer will require 10^8 seconds to compute all similarities (which is more than three years). The space required for explicitly storing the similarity matrix is of order of 10^6 TB. The vanilla method for embedding extraction can, therefore, be impractical.

It is possible to extract embeddings approximately by materializing only a subset of the entries in the similarity matrix $S = [s_{ij}]$. The key point is that there are large correlations between the entries in S , as a result of which the similarity matrix has a lot of built-in redundancy. For example, if both s_{ij} and s_{ik} are very large, then it is often the case that s_{jk} is very large as well. In mathematical terms, this observation also amounts to the fact that the matrix S often has much lower rank than its physical dimensionality n . With such redundancies, it is often possible to extract a lower-dimensional embedding of S approximately with sampling methods — this is a form of compression. We will discuss two solutions for achieving this goal. The first solution is a row-wise sampling approach that modifies the eigendecomposition technique already discussed. The second solution is a stochastic gradient-descent method that builds on the matrix factorization models of Chapter 8.

9.3.1 Nyström Sampling

The Nyström approach speeds up the embedding process by subsampling a subset of data objects and constructing the similarity matrix only on this (small) subset. The prototype embedding model is created using only this subset, and then it is generalized to out-of-sample points using some tricks from linear algebra [133].

The first step is to sample a set of p objects, and an in-sample similarity matrix S_{in} of size $p \times p$ is constructed in which the (i, j) th entry is the similarity between the i th and j th in-sample objects. Similarly, an $n \times p$ similarity matrix S_a is constructed in which the (i, j) th entry is the similarity between the i th object with the j th *in-sample* object. Note that the matrix S_{in} is contained within S_a , since each row of S_{in} is also a row of S_a . Then, the following pair of steps is used to first generate the embeddings of the in-sample points and then generalize the in-sample embeddings to all points (including out-of-sample points):

- **(In-sample embedding):** Diagonalize $S_{in} = Q\Sigma^2Q^T$. If there are fewer than p non-zero eigenvectors, then extract all $k < p$ non-zero eigenvectors in the $n \times k$ matrix Q_k and $k \times k$ diagonal matrix Σ_k . This step requires $O(p^2 \cdot k)$ time and $O(p^2)$ space. Since p is typically a small constant of the order of a few thousand, this step is extremely fast and space-efficient irrespective of the base number of objects.
- **(Universal embedding):** Let U_k denote the unknown $n \times k$ matrix containing the k -dimensional representation of the all n points in its rows. Although we already know

the embeddings of the in-sample points, we will use the properties of the similarity matrix in transformed space to derive all rows in a uniform way. The dot products of the n points in U_k and in-sample points in $Q_k \Sigma_k$ can be computed as the matrix product of U_k and $(Q_k \Sigma_k)^T$. This set of $n \times p$ dot products is contained in the matrix S_a , because it is assumed that S_a contains the dot products of embedded representations of all points and in-sample points. Therefore, we have the following:

$$S_a \approx \underbrace{U_k (Q_k \Sigma_k)^T}_{\text{Dot Products}} \quad (9.1)$$

The approximation is caused by the fact that the embedding of *all* points might require n -dimensional space, whereas we have restricted ourselves to at most p dimensions defined by the in-sample points. By postmultiplying each side with $Q_k \Sigma_k^{-1}$ and using $Q_k^T Q_k = I_k$, we obtain the following:

$$U_k \approx S_a Q_k \Sigma_k^{-1} \quad (9.2)$$

Therefore, we have an embedding of all n points in k -dimensional space. This step requires a simple matrix multiplication in time $O(n \cdot p \cdot k)$, which is linear in the number of objects in the data set.

It is noteworthy that the p in-sample rows in U_k are the same as the p rows in $Q_k \Sigma_k$.

It is interesting to note that we are able to represent n points in at most p dimensions, whereas the data-specific feature space for the full data might have dimensionality as large as n . What we have effectively done is to use the fact that a hyperplane defined by p points (and the origin) in feature space is an at most p -dimensional projection of the n -dimensional data-specific feature space, and it can be represented in at most $k \leq p$ coordinates. Therefore, we first find the exact k -dimensional representation of a subset of p points (where $k \leq p$); then, we project the remaining $(n - p)$ points from n -dimensional feature space to the k -dimensional subspace in which the p points lie. Therefore, the remaining $(n - p)$ points lose some accuracy of representation, which is expected in a sampling method. In fact, it is even possible to drop some of the smaller non-zero eigenvectors from the in-sample embedding for better efficiency.

9.3.2 Matrix Factorization with Stochastic Gradient Descent

A second approach is to use stochastic gradient descent in conjunction with the optimization model introduced in Section 9.2.4. Stochastic gradient descent performs the updates by sampling entries from the similarity matrix. Since the entries in the similarity matrix are highly correlated, it means that the matrix is often of approximately low rank. In such cases, it is possible to learn the top- k components of the embedding, by minimizing the squared sum of the residual (noise) entries. This is a similar approach to matrix factorization in recommender systems (cf. Section 8.3.2 of Chapter 8), where a small subset of matrix entries is sufficient to learn the factor matrices.

For ease in discussion, we will assume that a tiny subset of entries of the similarity matrix is materialized up front in the same way as a tiny subset of entries is available in a ratings matrix in recommender systems (cf. Section 8.3.2 of Chapter 8). In practice, one can always compute the similarity values on-the-fly for stochastic gradient descent, although fixing the “observed” entries up front also allows the use of vanilla gradient descent with

sparse matrix multiplications. Let $S = [s_{ij}]$ be an $n \times n$ similarity matrix, in which only a subset O of entries are observed:

$$O = \{(i, j) : s_{ij} \text{ is "observed"}\} \quad (9.3)$$

One can assume that the matrix S is symmetric, and therefore the observed set of similarities O can be grouped into symmetric pairs of entries satisfying $s_{ij} = s_{ji}$. It is desired to learn an $n \times k$ embedding U for user-specified rank k , so that for any observed entry (i, j) the dot product of the i th row of U and the j th row of U is as close as possible to the (i, j) th entry, s_{ij} , of S . In other words, the value of $\|S - UU^T\|_F^2$ should be as small as possible for the observed entries in S . This problem can be formulated only over the “observed” entries in O as follows:

$$\text{Minimize } J = \frac{1}{2} \sum_{(i,j) \in O} (s_{ij} - \sum_{p=1}^k u_{ip}u_{jp})^2 + \frac{\lambda}{2} \sum_{i=1}^n \sum_{p=1}^k u_{ip}^2$$

Therefore, we have changed the optimization model of Section 9.2.4, so that it is formulated only over a tiny subset of entries in S . Furthermore, regularization becomes particularly important in these cases as the subset of entries to be used is small. This problem is similar to the determination of factors in recommendation problems, and is a natural candidate for gradient-descent methods. The main difference is that the factorization is symmetric.

Let $e_{ij} = s_{ij} - \sum_{p=1}^k u_{ip}u_{jp}$ be the error of any entry (i, j) from set O at a particular value of the parameter matrix U . On computing the partial derivative of J with respect to u_{im} , one obtains the following:

$$\begin{aligned} \frac{\partial J}{\partial u_{im}} &= \sum_{j:(i,j) \in O} (s_{ij} + s_{ji} - 2 \cdot \sum_{p=1}^k u_{ip}u_{jp})(-u_{jm}) + \lambda u_{im} & \forall i \in \{1 \dots n\}, m \in \{1 \dots k\} \\ &= \sum_{j:(i,j) \in O} (e_{ij} + e_{ji})(-u_{jm}) + \lambda u_{im} & \forall i \in \{1 \dots n\}, m \in \{1 \dots k\} \\ &= -2 \sum_{j:(i,j) \in O} e_{ij}u_{jm} + \lambda u_{im} & \forall i \in \{1 \dots n\}, m \in \{1 \dots k\} \end{aligned}$$

Note that s_{ij} and s_{ji} are either both present or both absent from the observed entries because of the symmetric assumption. It is possible to express these partial derivatives in matrix form. Let $E = [e_{ij}]$ be an error matrix, in which (i, j) th entry is set to the error for any observed entry (i, j) in O , and 0, otherwise. When a small number of entries are observed, this matrix is a sparse matrix. It is not difficult to see that the entire $n \times k$ matrix of partial derivatives $\left[\frac{\partial J}{\partial u_{im}} \right]_{n \times k}$ is given by $-2EU$. This suggests that one should randomly initialize the matrix U of parameters, and use the following gradient-descent steps:

$$U \leftarrow U(1 - \alpha\lambda) + 2\alpha EU \quad (9.4)$$

Here, $\alpha > 0$ is the step size, which one can follow through to convergence. Note that the error matrix E is sparse, and therefore it makes sense to compute only those entries that are present in O before converting to a sparse data structure.

To determine the optimal rank k of the factorization, one can hold out a small subset $O_1 \subset O$ of the observed entries, which are not used for learning U . These entries are used to test the squared error $\sum_{(i,j) \in O_1} e_{ij}^2$ of the matrix U learned using various values of k . The

value of k at which the error of the held out entries is minimized is used. Furthermore, one can also use the held-out entries to determine the stopping criterion for the gradient-descent approach. The gradient-descent procedure is terminated when the error on the held-out entries begins to rise. The recovered matrix U provides a k -dimensional embedding of the data, which can be used in conjunction with machine learning algorithms.

The use of a fixed set of pre-computed entries in O allows the leveraging of gradient-descent methods. On the other hand, if we use *stochastic* gradient descent, we can simply sample any position in S and compute the similarity value on the fly. This type of approach does have the advantage that one does not have to cycle through the same set of entries in O . Presumably, the number of entries in the similarity matrix is so large that even when one samples as many entries as possible (with replacement) for stochastic gradient descent, most entries would not be visited more than once (or at all). Therefore, the stochastic gradient descent step boils down to the following step, which is executed repeatedly:

Randomly sample index pair $[i, j]$ and compute similarity value s_{ij} ;
 Compute the error $e_{ij} = s_{ij} - \sum_{p=1}^k u_{ip}u_{jp}$;
 Update $u_{im}^+ \leftarrow u_{im}(1 - \alpha\lambda) + 2e_{ij}u_{jm}$ for all $m \in \{1 \dots k\}$;
 Update $u_{jm}^+ \leftarrow u_{jm}(1 - \alpha\lambda) + 2e_{ij}u_{im}$ for all $m \in \{1 \dots k\}$;
 Update $u_{im} \leftarrow u_{im}^+$ and $u_{jm} \leftarrow u_{jm}^+$;

The similarity values are computed on the fly, as entries are sampled. The algorithm can be used even for similarity matrices that are not positive semidefinite. The diagonal entries will be learned automatically to create the closest positive semidefinite approximation.

Problem 9.3.1 Let S be an $n \times n$ symmetric matrix that is not positive semidefinite. It has $r \ll n$ negative eigenvalues of sizes $\lambda_1 \dots \lambda_r$. Show that the objective function $J = \|S - UU^T\|_F^2$ is always at least $\sum_{p=1}^r \lambda_p^2$, irrespective of the value of k in the $n \times k$ matrix U . What is the minimum value of k at which this error is guaranteed?

9.3.3 Asymmetric Similarity Decompositions

The decomposition $S = UU^T$ of the similarity matrix S is a symmetric one. However, it is also possible to use the *asymmetric* decomposition $S \approx UV^T$. Here, S is an $n \times n$ matrix, whereas U and V are both $n \times k$ matrices. In such a case, one can use some combination of the i th row of U and the i th row of V to create the embedding of the i th data point. For example, one can concatenate the i th row of U and the i th row of V to create a $2k$ -dimensional embedding of the i th data point. In such a case, the updates become similar to recommender systems (cf. Chapter 8), with the error matrix defined as $E = S - UV^T$, and the updates defined as follows:

$$\begin{aligned} U &\leftarrow U + \underbrace{\alpha EV}_{\Delta U} \\ V &\leftarrow V + \underbrace{\alpha E^T U}_{\Delta V} \end{aligned}$$

How does one use the decomposition components to create the embedding? There are several choices; for example, one can use only V to create the embedding. However, one can also concatenate the i th row of U and the i th row of V to create a $2k$ -dimensional embedding of the i th object. Using both the matrices U and V recognizes the fact that rows and columns capture different aspects of the similarity between objects. For example, in an asymmetric

follower-follower link matrix, similarities in terms of followers is different from similarity in terms of followees. Alice and Bob might both be movie stars and be similar in terms of their followers, whereas Alice and John might belong to the same family and be similar in terms of who they follow (i.e., followees). Using both U and V is helpful in accounting for both types of similarities.

Another way of performing asymmetric decompositions is truncated SVD of rank- k :

$$S \approx Q_k \Sigma_k P_k^T \quad (9.5)$$

Here, Q_k is analogous to U and P_k is analogous to V , if the scaling factors in Σ_k are ignored.

Finally, if S is diagonalizable with real eigenvectors/eigenvalues, one can use straightforward eigendecomposition to extract the embedding:

$$S = U \Delta U^{-1} \quad (9.6)$$

The columns of U contain the eigenvectors, and they are not necessarily orthonormal. In such a case, one can extract the top- k columns of U (corresponding to the largest eigenvalues) to create a k -dimensional embedding. Asymmetric decompositions are particularly useful for asymmetric similarity matrices that arise in a number of real-world applications:

1. In a social network, one user might follow another (or like another user), but the “similarity” relationship might not be reciprocated. Similarly, hyperlinks between Webpages can be viewed as directed indicators of similarity.
2. Even an undirected graph might have an asymmetric similarity network, if the edge weights are normalized in an asymmetric way. As we will see in Chapter 10, an adjacency matrix of an undirected graph can be converted into a *stochastic transition matrix* by normalizing each row to sum to 1, and the right eigenvectors of this transition matrix provide an embedding, referred to as the Shi-Malik embedding [115]. On the other hand, the symmetric decomposition of a symmetric normalization of the same adjacency matrix leads to a related embedding known as the Ng-Jordan-Weiss embedding [98]. Both embeddings are different forms of spectral decomposition, which are used for applications like *spectral clustering* (see Section 10.5.1 of Chapter 10).

Asymmetric decompositions can be computed using any of the methods discussed in Chapter 8. Most of this chapter will focus on symmetric embeddings.

9.4 Linear Algebra Operations on Similarity Matrices

Many machine learning applications use basic statistical and geometric operations on a data matrix such as computing the mean/variance of a data matrix, centering it, normalizing the data points, computing pairwise Euclidean distances (instead of dot product similarities), and so on. These operations are relatively easy to perform, if one already had access to the multidimensional data set. However, what if one was only provided the similarities? Would it be possible to perform these operations *indirectly* by modifying or using the similarity matrix (rather than the points)? These types of basic operations often turn out to be very useful in various machine learning applications.

Consider an $n \times n$ similarity matrix S based on n objects $o_1 \dots o_n$. These objects could be arbitrary types of objects such as time-series, sequences, and so on. Each object o_i has a

multidimensional embedding $\Phi(o_i)$, so that the (i, j) th entry s_{ij} in matrix S is defined by the following dot product:

$$s_{ij} = \Phi(o_i) \cdot \Phi(o_j)$$

With these notations, we will define the basic operations between two points:

9.4.1 Energy of Similarity Matrix and Unit Ball Normalization

The squared norm of the multidimensional representation $\Phi(o_i)$ of the i th object o_i , is computed in terms of similarities as follows:

$$\|\Phi(o_i)\|^2 = \Phi(o_i) \cdot \Phi(o_i) = s_{ii}$$

The above computation is defined for the *squared* norm. The norm is simply $\sqrt{s_{ii}}$. The total energy $E(S)$ of the data set is the sum of the squared norms of the points, which is simply the trace of the similarity matrix:

$$E(S) = \sum_{i=1}^n s_{ii} = \text{tr}(S)$$

In other words, the total energy in the data set is equal to the sum of the diagonal entries of the similarity matrix!

The norm can be used to normalize a similarity matrix, so that all engineered points $\Phi_n(o_i)$ lie on a unit ball. Note that dot products become *cosine* similarities for unit normalized points. Unlike dot products, cosine similarities are invariant to normalization. Consider the case where we have an unnormalized similarity matrix $S = [s_{ij}]$ corresponding to engineered representation $\Phi(\cdot)$, and we want to normalize these points to $\Phi_n(\cdot)$ on the unit ball.

$$\Phi_n(o_i) \cdot \Phi_n(o_j) = \text{cosine}[\Phi_n(o_i), \Phi_n(o_j)] = \frac{\Phi(o_i) \cdot \Phi(o_j)}{\|\Phi(o_i)\| \cdot \|\Phi(o_j)\|} = \frac{s_{ij}}{\sqrt{s_{ii}} \sqrt{s_{jj}}}$$

Each entry s_{ij} is replaced with the above normalized value. Note that a normalized similarity matrix will contain only 1s along the diagonal. This is because one has effectively normalized the data-specific kernel features to lie on a unit ball in $\mathcal{R}^n$.

9.4.2 Norm of the Mean and Variance

It is possible to compute the norm of the *mean* of a data set as follows:

$$\|\bar{\mu}\|^2 = \left\| \sum_{i=1}^n \Phi(o_i) / n \right\|^2 = \sum_{i=1}^n \sum_{j=1}^n \Phi(o_i) \cdot \Phi(o_j) / n^2 = \sum_{i=1}^n \sum_{j=1}^n s_{ij} / n^2$$

In other words, the squared norm of the mean is equal to average value of the entries in the similarity matrix. This value is always nonnegative according to Problem 9.2.3.

The total variance $\sigma^2(S)$ of the data set (over all dimensions) in the embedded space (induced by similarity matrix S) is obtained by subtracting the squared norm of the mean from the normalized energy (i.e., energy averaged over number of dimensions):

$$\sigma^2(S) = \text{Energy}(S) / n - \bar{\mu}^2 = \sum_{i=1}^n s_{ii} / n - \sum_{i=1}^n \sum_{j=1}^n s_{ij} / n^2$$

Note that the variance is the difference between the average diagonal entry and average matrix entry.

Problem 9.4.1 *The variance of a data set containing n points can be shown to be proportional to the sum of squared pairwise distances between points (over all $\binom{n}{2}$ pairs). Use this result to show that the variance $\sigma^2(S)$ in the data induced by similarity matrix S can be expressed in the following form for appropriately chosen n -dimensional vectors $\bar{y}_r$ for $r \in \{1, 2, \dots, n(n-1)/2\}$:*

$$\sigma^2(S) \propto \sum_{r=1}^{n(n-1)/2} \bar{y}_r^T S \bar{y}_r$$

The above problem also makes it evident why the variance will be nonnegative, given that the similarity matrix S is positive semidefinite.

9.4.3 Centering a Similarity Matrix

In some applications like PCA, it is assumed that the data is mean-centered. Unfortunately, there is no guarantee that the embedding induced by an arbitrary similarity matrix is mean-centered. Consider the case, where the data set $D = Q\Sigma$ can be extracted from the similarity matrix $S = Q\Sigma^2Q^T$. Therefore, the similarity matrix S can also be expressed as DD^T . Let M be an $n \times n$ matrix of 1s. Then, the centered version D_c of D can be expressed as follows:

$$D_c = (I - M/n)D \quad (9.7)$$

Then, the centered version S_c of similarity matrix S is given by $S_c = D_c D_c^T$. This similarity matrix can be expressed in terms of S as follows:

$$\begin{aligned} S_c &= D_c D_c^T = [(I - M/n)D][(I - M/n)D]^T \\ &= (I - M/n) \underbrace{(DD^T)}_S (I - M/n)^T = (I - M/n)S(I - M/n) \end{aligned}$$

Data matrices are often centered in machine learning as a preprocessing step to various tasks. A specific example is kernel PCA. It is also possible to recognize when a similarity matrix is mean-centered as follows:

Observation 9.4.1 (Recognizing Mean-Centered Similarities) *The sum of the i th row (or column) of a similarity matrix is the dot product between the embedding of the i th point and the sum of all vectors in the embedding. Therefore, all rows and columns of a **mean-centered** similarity matrix sum to 0.*

9.4.3.1 Application: Kernel PCA

The above approach provides a route to kernel PCA [112]. In kernel PCA, the $n \times n$ similarity matrix S is first centered to $S_c = (I - M/n)S(I - M/n)$, and subsequently it is diagonalized as $S_c = Q\Sigma^2Q^T$, which can be further expressed as the symmetric factorization $(Q\Sigma)(Q\Sigma)^T$. The embedding matrix is, therefore, given by $Q\Sigma$:

Definition 9.4.1 (Kernel PCA) *Let S be a positive semidefinite similarity matrix of size $n \times n$, which can be centered to $S_c = (I - M/n)S(I - M/n)$. Here, M is an $n \times n$ matrix of 1s. The embedding $Q\Sigma$ extracted by the eigendecomposition $S_c = Q\Sigma^2Q^T$ is referred to as the **kernel PCA embedding** of the points. The matrix $Q\Sigma$ contains the kernel PCA embedding of the i th point in its i th row.*

Therefore, kernel PCA differs from kernel SVD in terms of the preprocessing of the similarity matrix, just as PCA differs from SVD in the preprocessing of the data matrix. The reader is advised to carefully compare the above definition with Definition 9.2.2.

9.4.4 From Similarity Matrix to Distance Matrix and Back

In many applications of machine learning, distance functions are available instead of similarity functions. Therefore, a natural question arises as to how (dot product) similarity matrices can be converted into distance matrices and back.

It is easier to convert similarity matrices into distance matrices rather than the reverse. In order to understand this point, note that the Euclidean distance is closely related to the dot product similarity as follows:

$$\|\bar{X} - \bar{Y}\|^2 = \bar{X} \cdot \bar{X} + \bar{Y} \cdot \bar{Y} - 2\bar{X} \cdot \bar{Y}$$

Therefore, given a dot product matrix, it is relatively easy to create a squared Euclidean distance matrix using the above relationship for each entry. If δ_{ij} is the Euclidean distance between points i and j , then one can express δ_{ij} in terms of the entries of $S = [s_{ij}]$ as follows:

$$\delta_{ij}^2 = s_{ii} + s_{jj} - 2s_{ij}$$

The above relationship can also be expressed in matrix form. Let $\bar{1}_n$ be an n -dimensional column vector of 1s, and $\bar{z} = [s_{11}, s_{22}, \dots, s_{nn}]^T$. Let the matrix of squared distances be denoted by $\Delta = [\delta_{ij}^2]$. Then, Δ can be expressed in terms of similarities as follows:

$$\Delta = \bar{1}_n \bar{z}^T + \bar{z} \bar{1}_n^T - 2S \quad (9.8)$$

Note that the first two matrices on the right-hand side of the above summation are defined by outer products of vectors.

One can also compute the squared distance matrix from the similarity matrix. Let $\Delta = [\delta_{ij}^2]$ be the squared distance matrix. Then, the similarity between any pair of points $\bar{X}$ and $\bar{Y}$ can be expressed in terms of distances as follows:

$$\bar{X} \cdot \bar{Y} = \frac{1}{2} (\|\bar{X}\|^2 + \|\bar{Y}\|^2 - \|\bar{X} - \bar{Y}\|^2) \quad (9.9)$$

The squared norms of the individual points $\bar{X}$ and $\bar{Y}$ create a challenge for expressing the problem in terms of distances. The squared norms represent squared distances from the origin and the distance matrix will typically not contain any information about distances of points from the origin. Here, it is important to understand that the pairwise distances are invariant to origin translation, whereas dot product similarity is not. In other words, the similarity matrix will depend on which point we choose as the origin! A natural choice is to assume that the similarity matrix to be extracted is mean centered. In such a case, the squared norm of a data point becomes the squared distance of the point from the mean. Therefore, we make the following claim:

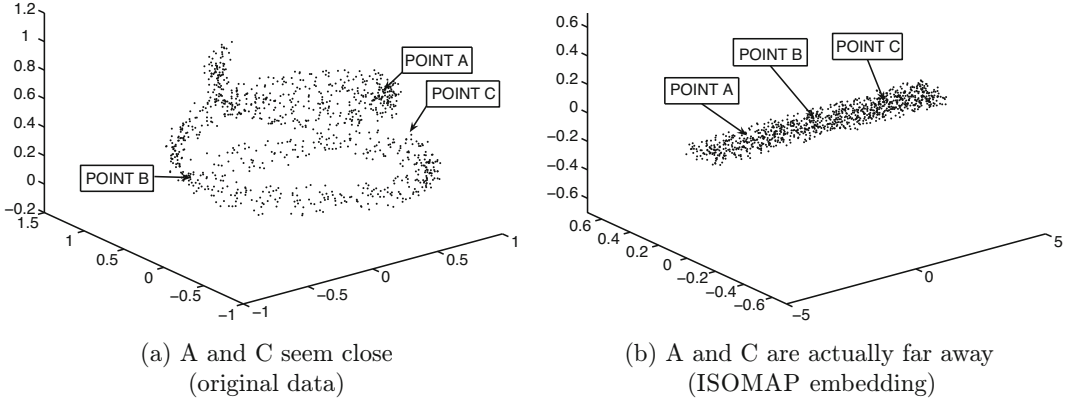
Lemma 9.4.1 *Let $\Delta = [\delta_{ij}^2]$ be an $n \times n$ squared distance matrix between embedded data points. Then, the mean-centered similarity matrix of dot products is given by the following:*

$$S = -\frac{1}{2} \left(I - \frac{M}{n} \right) \Delta \left(I - \frac{M}{n} \right)$$

Here, M is an $n \times n$ matrix of 1s.

Proof: When the similarity matrix S is mean-centered, Observation 9.4.1 implies that $MS = SM = 0$, when M is a matrix of 1s. Therefore, one can show the following:

$$S = \left(I - \frac{M}{n} \right) S \left(I - \frac{M}{n} \right) \quad (9.10)$$

Figure 9.1: Impact of *ISOMAP* embedding on distances

One can left-multiply and right-multiply both sides of Equation 9.8 with $(I - M/n)$ to obtain the following:

$$\left(I - \frac{M}{n}\right) \Delta \left(I - \frac{M}{n}\right) = \left(I - \frac{M}{n}\right) [\bar{\mathbf{I}}_n \bar{\mathbf{z}}^T + \bar{\mathbf{z}} \bar{\mathbf{I}}_n^T - 2S] \left(I - \frac{M}{n}\right)$$

Note that $M\bar{\mathbf{I}} = n\bar{\mathbf{I}}$. As a result, it is easy to show that $(I - \frac{M}{n})\bar{\mathbf{I}}_n = \bar{\mathbf{0}}$ and $\bar{\mathbf{I}}_n^T(I - \frac{M}{n}) = \bar{\mathbf{0}}^T$. We can use these results to simplify the above equation as follows:

$$\begin{aligned} \left(I - \frac{M}{n}\right) \Delta \left(I - \frac{M}{n}\right) &= -2 \left(I - \frac{M}{n}\right) [S] \left(I - \frac{M}{n}\right) \\ &= -2S, \quad [\text{Using Equation 9.10}] \end{aligned}$$

One can divide both sides by -2 to obtain the desired result. ■

The conversion of distance matrices to similarity matrices is more useful because it enables the use of kernel methods when distances are available. For example, in time-series data, some domain-specific methods provide distances instead of similarities to begin with. In such cases, the technique of *multidimensional scaling (MDS)* is used to create an embedding. Starting with the squared distance matrix Δ , it is converted to a centered similarity matrix using the approach discussed above. The large eigenvectors of this similarity matrix are used to create the embedding.

Problem 9.4.2 (Almost Negative Semi-definite Distance Matrix) We know that a valid similarity matrix using dot products must be positive semidefinite. A matrix is **almost negative semidefinite**, if it satisfies $\bar{\mathbf{y}}^T S \bar{\mathbf{y}} \leq 0$ for any **mean-centered** vector $\bar{\mathbf{y}}$. Show using this fact and Lemma 9.4.1 that any valid squared distance matrix in Euclidean space is almost negative semidefinite.

9.4.4.1 Application: ISOMAP

The *ISOMAP* approach is a great technique for straightening out curved manifolds in multidimensional space [126]. It can also compute *geodesic* distances and similarities, which

correspond to distances (and similarities) along a curved manifold rather than straight line distances. It can be argued that geodesic distances are more accurate representations of true distances as compared to straight-line distances in real applications. Such distances can be computed by using an approach that is derived from a non-linear dimensionality reduction and embedding method, known as *ISOMAP*. The approach consists of two steps:

1. Compute the k -nearest neighbors of each point. Construct a weighted graph G with nodes representing data points, and edge weights (costs) representing distances of these k -nearest neighbors.
2. For any pair of points $\bar{X}$ and $\bar{Y}$, report $Dist(\bar{X}, \bar{Y})$ as the shortest path between the corresponding nodes in G . Any graph-theoretic algorithm, such as the Dijkstra algorithm can be used [8].

Subsequently, a squared distance matrix is constructed. This distance matrix is converted into a similarity matrix using the approach of this section. Subsequently, the eigenvectors of this matrix are used to create the *ISOMAP* embedding. A 3-dimensional example is illustrated in Figure 9.1(a), in which the data is arranged along a spiral. In this figure, data points A and C seem much closer to each other than data point B. However, in the *ISOMAP* embedding of Figure 9.1(b), the data point B is much closer to each of A and C. This example shows how *ISOMAP* has a drastically different view of similarity and distances, as compared to the pure use of Euclidean distances.

9.5 Machine Learning with Similarity Matrices

There are two ways in which machine learning algorithms may be used, when a similarity matrix is provided instead of the data matrix. These two ways are as follows:

1. The similarity matrix S can be decomposed as $S = Q\Sigma^2Q^T$, and the embedding $Q\Sigma$ can be extracted. In some cases, only the top eigenvectors are retained, and the representation may be otherwise processed (e.g., whitening) to improve its quality. Subsequently, off-the-shelf machine learning algorithms are applied to the extracted representation.
2. Some algorithms in machine learning can be directly expressed in terms of similarities between points. An example is the SVM, in which the dual can be expressed in terms of dot products between points (cf. Section 6.4.4.1 of Chapter 6). In these cases, one can simply substitute the appropriate entry from the similarity matrix within the optimization function. This approach is referred to as the *kernel trick*.

The two choices provide equivalent solutions. Which one is preferable in practice? The general tendency in the machine learning community is to prefer the kernel trick. The reason is that the kernel trick is more space-efficient. However, explicit feature engineering also has a number of advantages. One can post-process the extracted features and discard the irrelevant ones. The lower-order features (i.e., smaller eigenvectors) can sometimes contain irrelevant noise, and explicit feature engineering methods tend to extract only the higher-order features. When using the kernel trick, one is effectively using *all* the features without any improvement/change including the irrelevant ones. When using feature engineering, tricks like whitening can also be used on the extracted features. In some problems like outlier detection, whitening is absolutely essential to create a high-quality implementation. The additional flexibility of feature engineering over the kernel trick is illustrated in Figure 9.2.

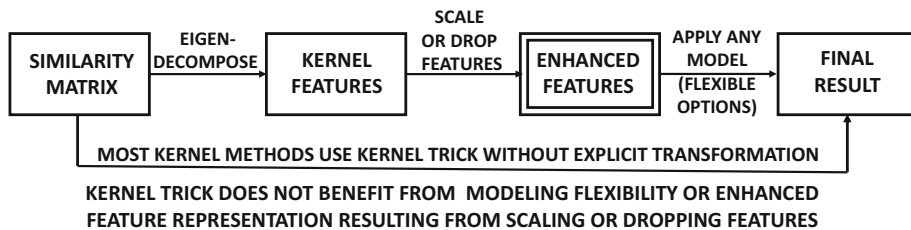


Figure 9.2: Two choices in algorithm design with kernel methods

As regards the issue of space- or time-efficiency, Nyström sampling or stochastic gradient descent can be used to efficiently extract the dominant features. In many cases, only the dominant features need to be extracted because lower-order features are not informative in most² applications. It is noteworthy that dominant feature extraction is far more efficient than the extraction of all features. The following sections discuss both feature engineering and the kernel trick.

9.5.1 Feature Engineering from Similarity Matrix

In this section, we will discuss algorithms for clustering, classification, and outlier detection with the use of feature engineering. One advantage of this approach is that it is very general, and one is not restricted to the use of a specific algorithm for clustering and classification.

9.5.1.1 Kernel Clustering

Imagine a setting, where you have an $n \times n$ similarity matrix S over n objects (e.g., chemical compounds). You would like to cluster these objects into similar groups. The broader approach of explicit feature engineering works by diagonalizing an $n \times n$ similarity matrix $S = Q\Sigma^2Q^T$ as follows:

Diagonalize $S = Q\Sigma^2Q^T$;
 Extract the n -dimensional embeddings in rows of $Q\Sigma$;
 Drop any zero columns from $Q\Sigma$ to create $Q_0\Sigma_0$;
 Apply any existing clustering algorithm on rows of $Q_0\Sigma_0$;

The columns of Q_0 contain the non-zero eigenvectors, and the n rows of $Q_0\Sigma_0$ contain the embeddings of the n points. Note that Q_0 is an $n \times r$ matrix and Σ_0 is an $r \times r$ matrix, since the zero rows *and* columns of Σ are removed to create Σ_0 . It is noteworthy that all n eigenvectors are extracted and only the zero eigenvectors are dropped. Such zero eigenvectors show up as zero columns in $Q\Sigma$. The embedding dimensionality can be as large as the number of points n , if no dimensions are dropped. The *space requirements* of such an approach can therefore be $O(n^2)$. Furthermore, the running time requirement for extracting all n eigenvectors is $O(n^3)$, which can be prohibitive. In many cases, one can use a dimensionality of the embedding that is far less than n . Furthermore, some implementations use the matrix Q to generate the embedding rather than $Q\Sigma$. Such an approach can be viewed as an indirect form of whitening (cf. Section 7.4.7 of Chapter 7). A specific example of a kernel clustering method that uses this form of whitening is *spectral clustering*, which is discussed in Section 10.5 of Chapter 10.

²An exception is outlier detection.

Since lower-order eigenvectors are often dropped anyway, it is possible to use any sampling method that preserves information only about dominant eigenvectors. A specific example is Nystrom sampling, which subsamples a set of s objects in order to create an s -dimensional representation. Typically, the value of s is independent of the data set size, although it depends on the complexity of the underlying data distribution (e.g., number of clusters). Then, the approach proceeds as follows:

- Draw a subsample of s objects from the data set;
- Use the Nystrom method (cf. Section 9.3.1) to create an s -dimensional representation of all objects denoted by the $n \times s$ matrix U_s ;
- Apply any existing clustering algorithm on U_s ;

It is also possible to use stochastic gradient descent (cf. Section 9.3.2) to extract the embedding matrix U_s . Furthermore, sampling-based methods can often be repeated to create multiple models. The averaged model from these multiple models is referred to as an *ensemble*, and it provides superior results.

9.5.1.2 Kernel Outlier Detection

The Mahalanobis method discussed in Section 7.4.7 can be generalized to the *kernel* Mahalanobis method. In the kernel Mahalanobis method, the Mahalanobis method is applied to an engineered representation of the data [5]. Note that the feature engineering approach already extracts a normalized SVD from the similarity matrix. Technically, the Mahalanobis method requires centering of the similarity matrix up front, although it makes no practical difference even if the uncentered matrix is used. Given an $n \times n$ similarity matrix S , the kernel Mahalanobis method works as follows:

- Diagonalize $S = Q\Sigma^2Q^T$;
- Extract the n -dimensional embeddings in rows of $Q\Sigma$;
- Drop any zero columns from $Q\Sigma$ to create $Q_0\Sigma_0$;
- Report the outlier score of each row of Q_0 as the Euclidean distance of that row from the mean computed over all rows of Q_0 ;

Here, it is noteworthy that we are using Q_0 rather than $Q_0\Sigma_0$ in order to compute the outlier score of each point. This is particularly important in outlier detection, because outliers are often hidden in the deviations along lower-order singular vectors. Multiplying with Σ_0 would de-emphasize such outliers. These forms of whitening and feature post-processing are not possible when using methods like the kernel trick (which is equivalent to always using $Q\Sigma$). Interestingly, some of the methods that do not use this type of whitening, such as the one-class SVM [113], are known to have weak performance [42].

Problem 9.5.1 Write the pseudocode for kernel outlier detection with Nystrom sampling.

9.5.1.3 Kernel Classification

Consider the case where we want to implement a kernel SVM. Assume that the $n \times n$ similarity matrix on the training objects is denoted by S . In addition, we have t test objects, and therefore we have the $t \times n$ matrix of test-training similarities denoted by S_t . Therefore, each row of S_t contains the similarity of a test object to all training objects. The columns of S_t are sorted in the same order of training objects as S .

The similarity matrix of training data is diagonalized as $S = Q\Sigma^2Q^T$. One can drop the zero columns of $Q\Sigma$ to yield the $n \times r$ matrix $U_0 = Q_0\Sigma_0$ with $r \leq n$ dimensions. Note that Q_0 is an $n \times r$ matrix, whereas Σ_0 is an $r \times r$ diagonal matrix with only non-zero

diagonal entries (singular values) of S . Therefore, Σ_0 is invertible. The rows of U_0 contain the *explicit* transformations of the training objects. The t *out-of-sample* objects in the test data can also be projected into this r -dimensional representation U_{test} by using the same trick as used in Nyström sampling:

$$\underbrace{S_t}_{t \times n} = U_{test} U_0^T = \underbrace{U_{test}}_{t \times r} \underbrace{(Q_0 \Sigma_0)^T}_{r \times n} \quad (9.11)$$

The above relationship is a result of the fact that the dot product of the rows in U_{test} and U_0 correspond to test-training similarities. Multiplying both sides with $Q_0 \Sigma_0^{-1}$ and using $Q_0^T Q_0 = I$ on the left-hand side, we obtain:

$$U_{test} = S_t Q_0 \Sigma_0^{-1} \quad (9.12)$$

The matrix U_{test} contains the engineered representations of the test objects. Therefore, we present the algorithm for kernel SVMs as follows:

```

Diagonalize  $S = Q \Sigma^2 Q^T$ ;
Extract the  $n$ -dimensional embedding in rows of  $Q \Sigma$ ;
Drop any zero eigenvectors from  $Q \Sigma$  to create  $Q_0 \Sigma_0$ ;
{ The  $n$  rows of  $Q_0 \Sigma_0$  and their class labels constitute training data }
Apply linear SVM on  $Q_0 \Sigma_0$  and class labels to learn model  $\mathcal{M}$ ;
Convert test-train similarity matrix  $S_t$  to representation matrix  $U_{test}$  using Equation 9.12;
Apply  $\mathcal{M}$  on each row of  $U_{test}$  to yield predictions;

```

The above implementation is *identical* to the kernel SVM that is implemented using the kernel trick (cf. Section 9.5.2.1). *One can substitute the SVM with any learning algorithm like logistic regression or least-squares classification*, which is one of the advantages of explicit feature engineering. One can also use this approach in conjunction with Nyström sampling in order to improve efficiency.

Problem 9.5.2 *Show how the kernel SVM approach discussed in this section can be efficiently implemented with Nyström sampling.*

9.5.2 Direct Use of Similarity Matrix

The direct use of similarity matrices to implement machine learning algorithms (without extracting the embedding as an intermediate step) is referred to as the *kernel trick*. Although the kernel trick is often touted as the only practical way of implementing these algorithms, this is not precisely an accurate view. Explicit feature engineering has several benefits, the most important of which is the fact that one can modify or normalize features in intermediate steps. In some applications like outlier detection, this is so important that the variants using the kernel trick are not quite as effective [5, 42].

9.5.2.1 Kernel K-Means

Let $S = [s_{ij}]$ be an $n \times n$ similarity matrix, which contains the pairwise similarity information between the objects $o_1 \dots o_n$. These objects may be of any arbitrary type (e.g., sequences or chemical compound graphs). Assume that the embedding implied by the kernel similarity matrix is denoted by $\Phi(\cdot)$ so that $s_{ij} = \Phi(o_i) \cdot \Phi(o_j)$.

The kernel k -means algorithm proceeds as follows. We start with a random assignment of points to the k clusters, denoted by $\mathcal{C}_1 \dots \mathcal{C}_k$. The usual implementation of the k -means

algorithm determines the centroids of the clusters as the representatives of the next iteration. The kernel k -means algorithm computes the dot product of each point to the various cluster centroids *in transformed space* and re-assigns each point to its closest centroid in the next iteration. How can one compute the dot product between an embedded object $\Phi(o_i)$ and the centroid $\bar{\mu}_j$ of $\mathcal{C}_j$ (in transformed space)? This can be achieved as follows:

$$\Phi(o_i) \cdot \bar{\mu}_j = \Phi(o_i) \cdot \frac{(\sum_{q \in \mathcal{C}_j} \Phi(o_q))}{|\mathcal{C}_j|} = \frac{\sum_{q \in \mathcal{C}_j} \Phi(o_i) \cdot \Phi(o_q)}{|\mathcal{C}_j|} = \sum_{q \in \mathcal{C}_j} \frac{s_{iq}}{|\mathcal{C}_j|}$$

Therefore, for any given object o_i , we only need to compute its average kernel similarity to all points in that cluster. Instead of the centroids, the approach does require the explicit maintenance of assignments of each point to various clusters in order to recompute the assignments for the next iteration. As in all k -means algorithms, the approach is iterated to convergence. For a data set containing n points, the approach requires $O(n^2)$ time in each iteration of the k -means algorithm, which can be quite costly for large data sets. The approach also requires the computation of the entire kernel matrix, which might require $O(n^2)$ storage. However, if the similarity function can be computed efficiently, then one does not need to store the kernel matrix a priori, but simply recompute individual entries on the fly when they are needed.

This algorithm is *identical* to the approach discussed in Section 9.5.1.1, when the embedding $Q_0 \Sigma_0$ is used in Section 9.5.1.1 and the k -means algorithm is used in the final step. However, a disadvantage of the kernel trick is that it can be paired with only a restricted subset of clustering algorithms (e.g., k -means) that use similarity functions between points. Not all clustering algorithms are equally friendly to the use of the kernel trick. Furthermore, one can perform no further engineering or normalization of the extracted features, if they are being used only indirectly via the kernel trick.

9.5.2.2 Kernel SVM

As in other problems of this section, we assume that each object o_i has an engineered representation $\Phi(o_i)$. The similarities between the n objects are contained in the $n \times n$ similarity matrix S . As consistently used throughout this book, the class labels are denoted by $y_1 \dots y_n \in \{-1, +1\}$ for the n training instances. We simply copy the dual problem of the SVM from Section 6.4.4.1 as follows (in minimization form):

$$\begin{aligned} \text{Minimize } L_D &= \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i - \left\{ \sum_{i=1}^n \alpha_i \alpha_j y_i y_j \underbrace{\Phi(o_i) \cdot \Phi(o_j)}_{s_{ij}} \right\} \\ \text{subject to:} \\ 0 &\leq \alpha_i \leq C \quad \forall i \in \{1 \dots n\} \end{aligned}$$

The notations of this problem are the same as those in Section 6.4.4.1. Each α_i is the i th dual variable. The quantity C is the slack penalty. The *only difference* from the objective function of Section 6.4.4.1 is that we have replaced the training point $\bar{X}_i$ with an engineered point $\Phi(o_i)$. However, the dot product between $\Phi(o_i)$ and $\Phi(o_j)$ is simply s_{ij} , and therefore the engineered points can be made to disappear from the formulation and be replaced by similarities. The partial derivative of L_D with respect to α_k is as follows:

$$\frac{\partial L_D}{\partial \alpha_k} = y_k \sum_{q=1}^n y_q \alpha_q s_{kq} - 1 \quad \forall k \in \{1 \dots n\} \quad (9.13)$$

This is a convex optimization problem with box constraints. Therefore, one starts by setting the vector of Lagrangian parameters $\bar{\alpha} = [\alpha_1 \dots \alpha_n]$ to an n -dimensional vector of 0s and uses the following update steps with learning rate η :

```

repeat
  Update  $\alpha_k \leftarrow \alpha_k + \eta \left[ 1 - y_k \sum_{q=1}^n y_q \alpha_q s_{kq} \right]$  for each  $k \in \{1 \dots n\}$ ;
  { Update is equivalent to  $\bar{\alpha} \leftarrow \bar{\alpha} - \eta \left[ \frac{\partial L_D}{\partial \bar{\alpha}} \right]$  }
  for each  $k \in \{1 \dots n\}$  do begin
     $\alpha_k \leftarrow \min\{\alpha_k, C\}$ ;
     $\alpha_k \leftarrow \max\{\alpha_k, 0\}$ ;
  endfor;
until convergence

```

After the variables $\alpha_1 \dots \alpha_n$ have been learned, the test objects are predicted using their similarities with training objects. This is because the classification of an unseen test instance $\bar{Z}$ is given by the sign of $\bar{W} \cdot \Phi(\bar{Z})$, where our analysis in Chapter 6 shows that $\bar{W} = \sum_{j=1}^n \alpha_j \bar{y}_j \Phi(\bar{X}_j)$. Here, $\Phi(\bar{X}_j)$ is the engineered representation of the j th training point. Therefore, the prediction of $\bar{Z}$ is given by $\sum_{j=1}^n \alpha_j \bar{y}_j \Phi(\bar{X}_j) \cdot \Phi(\bar{Z})$. This is simply the weighted sum of the similarities of the training instances with the test instance, where the weight of the j th similarity is $y_j \alpha_j$.

One can also express this result in terms of training-test similarity matrices. Let S_t be the $t \times n$ similarity matrix of training-test similarities between test objects and training objects. Let $\bar{\gamma}$ be an n -dimensional column vector in which the j th component is $y_j \alpha_j$. Then, the analysis of the previous paragraph shows that the prediction of the t test instances is given by the sign of each element in the t -dimensional vector $S_t \bar{\gamma}$.

The optimization model discussed in this section is *identical* to the one discussed in Section 9.5.1.3 (although the computational procedures are very different in the two cases). However, the approach in Section 9.5.1.3 is more flexible, because it cleanly decouples feature engineering from model building. Therefore, it can be used more easily with any off-the-shelf classification model or computational procedure.

9.6 The Linear Algebra of the Representer Theorem

In this section, we will drop the use of $\Phi(\cdot)$ for brevity; the training points $\bar{X}_1 \dots \bar{X}_n$ and the test point $\bar{Z}$ already represent the engineered representations. Note that we could make the same arguments by adding the function $\Phi(\cdot)$ to each data point, but it will unnecessarily make our mathematical formulas cumbersome.

When using the kernel trick, it becomes essential to identify a computational procedure that uses similarities rather than individual data points. The *representer theorem* is a useful principle from linear algebra that provides a boiler plate method to convert many optimization formulations into one that uses similarities. The approach is applicable to optimization models satisfying the following two properties:

1. The parameters of the optimization problem can be expressed as one or more vectors in the same multidimensional space as the individual data points. For example, the weight vector $\bar{W}$ in problems like the SVM lies in the same multidimensional space as the data points.
2. The objective function of the optimization problem can be expressed as a function of one or more of (i) dot products between points, (ii) dot products between the

parameter vectors and points, and (iii) dot products between the parameter vectors themselves (e.g., L_2 -regularizer).

Under these circumstances, a representer theorem can be used to transform any machine learning problem on multidimensional vectors into a formulation that uses only similarities between points. Interestingly, all the linear classification models we have seen so far satisfy this property. In other words, the optimization formulation can be converted into one that uses only similarities between objects.

Consider the L_2 -regularized form of all linear models discussed in Chapter 4 over the training pairs $(\bar{X}_1, y_1) \dots (\bar{X}_n, y_n)$, where each $\bar{X}_i$ is a row vector. Furthermore, the prediction of y_i is done as $\hat{y}_i = f(\bar{W} \cdot \bar{X}_i^T)$ for some function $f(\cdot)$ that depends on the nature of the target variable (e.g., numeric, binary, or categorical). The loss function can be written as $L(y_i, \bar{W} \cdot \bar{X}_i^T)$ in each case, because the objective function compares $\bar{W} \cdot \bar{X}_i^T$ to y_i in each case to decide the loss. The overall objective function, including the regularizer, may be written as follows:

$$\text{Minimize } J = \sum_{i=1}^n L(y_i, \bar{W} \cdot \bar{X}_i^T) + \frac{\lambda}{2} \|\bar{W}\|^2 \quad (9.14)$$

Consider a situation in which the training data points have dimensionality d , but all of them lie on a 2-dimensional plane. Note that the optimal linear separation of points on this plane can always be achieved with the use of a 1-dimensional line on this 2-dimensional plane. Furthermore, this separator is more concise than any higher dimensional separator and will therefore be preferred by the L_2 -regularizer. A 1-dimensional separator of training points lying on a 2-dimensional plane is shown in Figure 9.3(a). Although it is also possible to get the same separation of training points using any 2-dimensional plane (e.g., Figure 9.3(b)) passing through the 1-dimensional separator of Figure 9.3(a), such a separator would not be preferred by an L_2 -regularizer because of its lack of conciseness. In other words, given a set of training data points (row vectors from a data matrix) denoted by $\bar{X}_1 \dots \bar{X}_n$, the separator $\bar{W}$, defined as a column vector, always lies in the space spanned by these vectors (after converting them to column vectors). We state this result below, which is a very simplified version of the representer theorem, and is specific to linear models with L_2 -regularizers.

Theorem 9.6.1 (Simplified Representer Theorem) *Let J be any optimization problem of the following form:*

$$\text{Minimize } J = \sum_{i=1}^n L(y_i, \bar{W} \cdot \bar{X}_i^T) + \frac{\lambda}{2} \|\bar{W}\|^2$$

Then, any optimum solution $\bar{W}^$ to the aforementioned problem lies in the subspace spanned by the training points $\bar{X}_1^T \dots \bar{X}_n^T$. In other words, there must exist real values $\beta_1 \dots \beta_n$ such that the following is true:*

$$\bar{W}^* = \sum_{i=1}^n \beta_i \bar{X}_i^T$$

Proof: Suppose that $\bar{W}^*$ cannot be expressed in the subspace spanned by the training points. Then, let us decompose $\bar{W}^*$ into the portion $\bar{W}_{\parallel} = \sum_{i=1}^n \beta_i \bar{X}_i^T$ spanned by the training points and an additional orthogonal residual $\bar{W}_{\perp}$. In other words, we have:

$$\bar{W}^* = \bar{W}_{\parallel} + \bar{W}_{\perp} \quad (9.15)$$

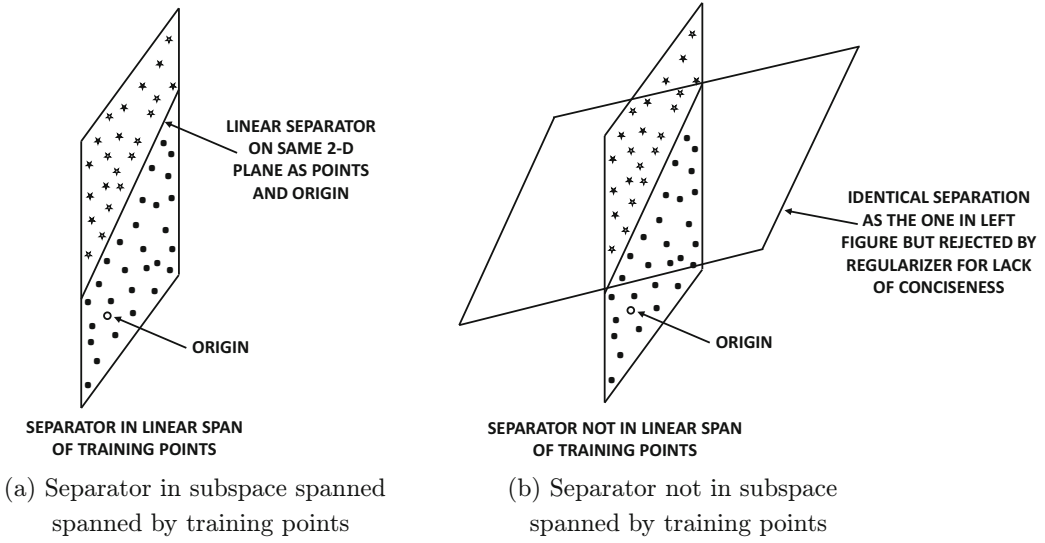


Figure 9.3: Both the linear separators in (a) and (b) provide exactly the same separation of training points, except that the one in (a) can be expressed as a linear combination of the training points. The separator in (b) will always be rejected by the regularizer. The key point of the representer theorem is that a separator $\bar{W}$ can always be found in the plane (subspace) of the training points with an identical separation to one that does not

Then, it suffices to show that $\bar{W}^*$ can be optimal only when $\bar{W}_\perp$ is the zero vector.

Each $(\bar{W}_\perp \cdot \bar{X}_i)$ has to be 0, because $\bar{W}_\perp$ is orthogonal to the subspace spanned by the various training points. The optimal objective J^* can be written as follows:

$$\begin{aligned}
 J^* &= \sum_{i=1}^n L(y_i, \bar{W}^* \cdot \bar{X}_i^T) + \frac{\lambda}{2} \|\bar{W}^*\|^2 = \sum_{i=1}^n L(y_i, (\bar{W}_\parallel + \bar{W}_\perp) \cdot \bar{X}_i^T) + \frac{\lambda}{2} \|\bar{W}_\parallel + \bar{W}_\perp\|^2 \\
 &= \sum_{i=1}^n L(y_i, \bar{W}_\parallel \cdot \bar{X}_i^T + \underbrace{\bar{W}_\perp \cdot \bar{X}_i^T}_0) + \frac{\lambda}{2} \|\bar{W}_\parallel\|^2 + \frac{\lambda}{2} \|\bar{W}_\perp\|^2 \\
 &= \sum_{i=1}^n L(y_i, \bar{W}_\parallel \cdot \bar{X}_i^T) + \frac{\lambda}{2} \|\bar{W}_\parallel\|^2 + \frac{\lambda}{2} \|\bar{W}_\perp\|^2
 \end{aligned}$$

It is noteworthy that $\|\bar{W}_\perp\|^2$ must be 0, or else $\bar{W}_\parallel$ will be a better solution than $\bar{W}^*$.

Therefore, $\bar{W}^* = \bar{W}_\parallel$ lies in the subspace spanned by the training points. ■

Intuitively, the representer theorem states that for a particular family of loss functions, one can always find an optimal linear separator within the subspace spanned by the training points (see Figure 9.3), and the regularizer ensures that this is the concise way to do it. After all, even though the embedding of an object might be infinite dimensional, each data object only lies in an n -dimensional projection of this space for a data set of size n . This n -dimensional projection is defined by the span of the n vectors $\bar{X}_1^T \dots \bar{X}_n^T$. The parameter vector $\bar{W}$ also lies in the span of this n -dimensional subspace; this is the essence of the representer theorem.

The representer theorem provides a boilerplate method to create an optimization model that is expressed as a function of dot products:

For any given optimization model of the form of Equation 9.14 plug in $\bar{W} = \sum_{i=1}^n \beta_i \bar{X}_i^T$ to obtain a new optimization problem parameterized by $\beta_1 \dots \beta_n$, and expressed only in terms of dot products between training points. Furthermore, the same approach is also used while evaluating $\bar{W} \cdot \bar{Z}^T$ for test instance $\bar{Z}$.

Consider what happens when one evaluates $\bar{W} \cdot \bar{X}_i^T$ in order to plug it into the loss function:

$$\bar{W} \cdot \bar{X}_i^T = \sum_{p=1}^n \beta_p \bar{X}_p^T \cdot \bar{X}_i^T = \sum_{p=1}^n \beta_p \bar{X}_p \cdot \bar{X}_i \quad (9.16)$$

Furthermore, the regularizer $\|\bar{W}\|^2$ can be expressed as follows:

$$\|\bar{W}\|^2 = \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j \bar{X}_i \cdot \bar{X}_j \quad (9.17)$$

In order to kernelize the problem, all we have to do is to substitute the dot product with the similarity value $s_{ij} = \bar{X}_i \cdot \bar{X}_j$ from the $n \times n$ similarity matrix S . Note that each $\bar{X}_i$ is really the embedded representation $\Phi(o_i)$ of an object. Therefore, one obtains the following optimization objective function:

$$J = \sum_{i=1}^n L(y_i, \sum_{p=1}^n \beta_p s_{pi}) + \frac{\lambda}{2} \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j s_{ij} \quad [\text{General form}]$$

In other words, all we need to do is to substitute each $\bar{W} \cdot \bar{X}_i^T$ in the loss function with $\sum_p \beta_p s_{pi}$. Therefore, one obtains the following form for least-squares regression:

$$J = \frac{1}{2} \sum_{i=1}^n (y_i - \sum_{p=1}^n \beta_p s_{pi})^2 + \frac{\lambda}{2} \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j s_{ij} \quad [\text{Least-squares regression}]$$

By substituting $\bar{W} \cdot \bar{X}_i^T = \sum_p \beta_p s_{pi}$ into the loss functions of various classifiers for binary data, one can obtain corresponding optimization formulations:

$$J = \sum_{i=1}^n \max\{0, 1 - y_i \sum_{p=1}^n \beta_p s_{pi}\} + \frac{\lambda}{2} \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j s_{ij} \quad [\text{SVM}]$$

$$J = \sum_{i=1}^n \log(1 + \exp(-y_i \sum_{p=1}^n \beta_p s_{pi})) + \frac{\lambda}{2} \sum_{i=1}^n \sum_{j=1}^n \beta_i \beta_j s_{ij} \quad [\text{Logistic Regression}]$$

These *unconstrained* optimization problems are conveniently expressed in terms of pairwise similarities, and parameterized by $\bar{\beta} = [\beta_1 \dots \beta_n]^T$. Any of the optimization procedures discussed in Chapter 4 can be used to learn these parameters.

How does one perform prediction with the use of similarities? Predictions in linear classifiers are done by using the dot products between test instances and $\bar{W}$. Consider the case in which we have a $t \times n$ matrix S_t of similarities between test-training pairs. The dot product of each test instance with $\bar{W}$ amounts to the dot product between the corresponding row of S_t and $\bar{\beta}^T$, as in the case of training instances (cf. Equation 9.16). This is because

$\overline{W}$ can be expressed as the summation $\sum_i \beta_i \overline{X}_i^T$ over all engineered training instances, and the dot product with the engineered test instance $\overline{Z}^T$ simply extracts the corresponding row from S_t ; the entries in that row are $\overline{Z}^T \cdot \overline{X}_i^T$. Therefore, the entire set of t test instances can be predicted as the t -dimensional vector $S_t \beta$. In the case of the classification problem, one needs to use the sign of each element of this vector as the predicted class label.

9.7 Similarity Matrices and Linear Separability

So far, we have justified the use of similarity matrices as an avenue for feature engineering of data types that are not multidimensional (e.g., chemical compounds). What happens when we have a multidimensional data set? Surely, it would not make sense to compute a dot product matrix and then extract features from the matrix via eigendecomposition — by doing so, one would only obtain a rotrelection of the original data set. Here, the key point is that changing the dot product to a more carefully designed similarity function has the effect of changing the data representation so that a simple model with inherent limitations (e.g., the limitation of being a linear model) works much more effectively. This is because by changing the similarity function, we are also changing its underlying multidimensional (engineered) representation to a nonlinear function of the original representation.

In classification and regression applications, one is often looking for feature representations such that the dependent variable is linearly related to the features. For example, in the case of an SVM, we are looking for the linear separator $\overline{W} \cdot \Phi(\overline{X}) = 0$, where $\Phi(\overline{X})$ is the engineered representation obtained by symmetric decomposition of the similarity matrix. Since $\Phi(\overline{X})$ depends on the similarity matrix, a natural question arises as to whether some similarity matrices are better than others in achieving linear separation between the classes. In many applications, an analyst might have little control over the similarity or distance matrix provided by a given application. For example, in a time-series application, the distance matrix may be defined by a dynamic time-warping application, which might be converted to a similarity matrix (cf. Section 9.4.4). However, the analyst still has the ability to *post-process* the matrix, so that its decomposition yields better features. In general, *higher embedding dimensionalities tend to make linear separation more likely*. The effective embedding dimensionality of an $n \times n$ similarity matrix is often much less than n because many eigenvalues are extremely small. The embedding dimensionality of a positive semidefinite similarity matrix can often be increased by applying a *superlinear function* on its entries. In other words, we apply an *element-wise superlinear function* $F(s_{ij})$ to each element s_{ij} of the similarity matrix S . The simplest elementwise function is the polynomial function:

$$F(s_{ij}) = (c + s_{ij})^h$$

In this case, c and h are nonnegative hyperparameters. For example, choosing $c = 1$ and $h = 2$ corresponds to adding 1 to each value in the similarity matrix and squaring it. *For certain types of functions, such as the above, the positive semidefiniteness of the similarity matrix is not lost by this change.*

If the original embedded data (implied by the similarity matrix) is such that one of the two classes lies inside the ellipse $x^2 + 4y^2 \leq 10$ and the other lies outside it, then performing this type of quadratic operation on the similarity matrix will lead to at least one two-dimensional projection in the embedding, where the elliptical boundary becomes a linear separator. This situation is shown in Figure 9.4, where two of the embedded dimensions are shown. In order to understand this point, we recommend the reader to work out the practice Problem 9.7.1 below.

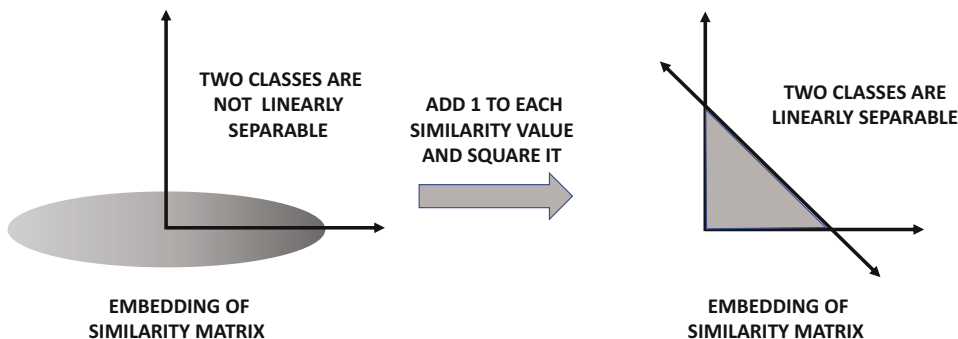


Figure 9.4: Applying an element-wise superlinear function on a similarity matrix often creates higher-dimensional embeddings in which points are linearly separable in carefully chosen projections: the relevant 2-dimensional projection of the embedding on the right is shown

Problem 9.7.1 Consider two points (x_1, y_1) and (x_2, y_2) in 2-dimensional space and the dot product similarity $s = x_1 \cdot x_2 + y_1 \cdot y_2$. Now imagine that you modify the similarity to the superlinear function $s' = (1 + s)^2$. Show that s' can be expressed as the dot product between $(x_1^2, y_1^2, x_1 y_1 \sqrt{2}, x_1 \sqrt{2}, y_1 \sqrt{2}, 1)$ and $(x_2^2, y_2^2, x_2 y_2 \sqrt{2}, x_2 \sqrt{2}, y_2 \sqrt{2}, 1)$.

Now consider a situation where an 100000×100000 similarity matrix S has two non-zero eigenvalues, and the 2-dimensional embedding $[x_1, x_2]$ extracted by eigendecomposition of S exhibits the property that all members of one class are lie inside the ellipse $x_1^2 + 4y_1^2 \leq 10$, whereas all members of the second class lie outside this ellipse. Discuss why the two classes will become linearly separable if the embedding is extracted by eigendecomposition of a modified similarity matrix S' in which we add 1 to each similarity entry and then square it. What is the dimensionality of the modified embedding (i.e., number of non-zero eigenvectors of S')?

There are two other functions that are commonly used for increasing the embedding dimensionality and capturing nonlinearity:

$$\begin{aligned} F(s_{ij}) &= \tanh(\kappa s_{ij} - \delta) && \text{[Sigmoid function]} \\ F(s_{ij}) &= \exp(s_{ij}/\sigma^2) && \text{[Gaussian function]} \end{aligned}$$

Of course, applying a superlinear function does not always help, because it could lead to overfitting. The level of sensitivity of the superlinear function depends on the parameters (such as the bandwidth, σ , of the Gaussian function above), which are often chosen in a data-driven manner. For example, one can test the classification accuracy on out-of-sample data in order to select σ^2 . A critical fact about many of these functions is that they do not destroy the positive semidefinite nature of the underlying similarity matrix. In Section 9.7.1, we will discuss some of these transformations.

The above ideas are used frequently for multidimensional data, where the similarity value s_{ij} is often set to a superlinear function of the dot product. Let $\bar{X}_1 \dots \bar{X}_n$ be the n points, and the similarity s_{ij} be defined by the kernel function $K(\bar{X}_i, \bar{X}_j)$. Then, the common kernel functions used for multidimensional data are defined in Table 9.1.

Table 9.1: Table of common kernel functions

Function	Form
Linear Kernel	$K(\bar{X}_i, \bar{X}_j) = \bar{X}_i \cdot \bar{X}_j$
Gaussian Radial Basis Kernel	$K(\bar{X}_i, \bar{X}_j) = \exp(-\ \bar{X}_i - \bar{X}_j\ ^2 / (2 \cdot \sigma^2))$
Polynomial Kernel	$K(\bar{X}_i, \bar{X}_j) = (\bar{X}_i \cdot \bar{X}_j + c)^h, c \geq 0$
Sigmoid Kernel	$K(\bar{X}_i, \bar{X}_j) = \tanh(\kappa \bar{X}_i \cdot \bar{X}_j - \delta)$

Each of the kernels in Table 9.1 has parameters associated with it, which need to be learned in a data-driven manner. Note that the above kernels are similar to the techniques discussed for modifying the similarity matrix. These modifications improve the level of separation among different classes. The dimensionality of the embedding depends on the nature of the kernel function. For example, the Gaussian kernel leads to an infinite-dimensional embedding to represent all possible data pairs in $\mathcal{R}^n \times \mathcal{R}^n$, although the data-specific embedding is always n -dimensional and can be materialized for a data set containing n points (using the eigendecomposition methods discussed earlier in this chapter).

9.7.1 Transformations That Preserve Positive Semi-definiteness

It is useful to understand the nature of transformations of a positive semidefinite matrix that preserve the positive semidefiniteness property. All the results below apply to the $n \times n$ positive semidefinite matrix $S = [s_{ij}]$, unless otherwise mentioned:

1. The matrix aS is positive semidefinite for $a > 0$. [Note that $\bar{x}^T(aS)\bar{x} = a[\bar{x}^T S \bar{x}] \geq 0$.]
2. If S_1 and S_2 are positive semidefinite, then $S_1 + S_2$ is positive semidefinite. [This is easy to prove by showing $\bar{x}^T(S_1 + S_2)\bar{x} \geq 0$ for all $\bar{x}$.]
3. An $n \times n$ matrix C containing the constant non-negative value c in each entry is positive semidefinite because $\bar{x}^T C \bar{x} = c(\sum_i x_i)^2 \geq 0$.
4. If S_1 and S_2 are positive semidefinite matrices of the same size, then $S_1 \odot S_2$ is positive semidefinite. Here, $\odot$ indicates entry-wise product. This result is referred to as *Schur's product theorem*, and the proof is nonobvious (see Problem 9.7.2).
5. The matrix $\overbrace{S \odot S \odot \dots \odot S}^{k \text{ times}}$ is positive semidefinite. [This is easy to show by applying the previous result recursively.]
6. Let $f(x)$ be a polynomial function with nonnegative coefficients that is applied to each entry of S . Then, the resulting matrix is positive semidefinite. [This is easy to show by combining four of the above results.]
7. The matrix $\exp(aS)$ is positive semidefinite for $a > 0$. Here $\exp(\cdot)$ refers to entry-wise exponentiation of the matrix. [This is easy to show by observing that an exponentiation can be expressed as an infinite polynomial with nonnegative coefficients using the Taylor expansion (cf. Equation 1.31 of Chapter 1). Therefore, each entry in the matrix is an infinite polynomial, and one reverts to the polynomial case above.]
8. Let $\delta_1 \dots \delta_n$ be n real values. Then, the scaled similarity matrix in which the (i, j) th entry is $\delta_i s_{ij} \delta_j$ is positive semidefinite. [Since $\bar{x}^T S \bar{x}$ is nonnegative for any $\bar{x}$, so is the value $[\bar{x} \odot \bar{\delta}]^T S [\bar{x} \odot \bar{\delta}]$. Here, we have $\bar{\delta} = [\delta_1 \dots \delta_n]^T$.]

Next, we list the Schur's product theorem as a practice exercise in a step-by-step manner, because it was used in one of the above results to show that $S_1 \odot S_2$ is positive semidefinite, when S_1 and S_2 are positive semidefinite.

Problem 9.7.2 (Schur's Product Theorem) *Let $S_1 = AA^T$ and $S_2 = BB^T$ be two positive semidefinite matrices. Let $\bar{a}_i$ the i th row of A and $\bar{b}_i$ be the i th row of B .*

- *Show that for any vector $\bar{x}$, one can express $\bar{x}^T(S_1 \odot S_2)\bar{x}$ in the following form:*

$$\bar{x}^T(S_1 \odot S_2)\bar{x} = \sum_i \sum_j x_i x_j [\bar{a}_i \bar{a}_j^T][\bar{b}_i \bar{b}_j^T]$$

- *Suppose that q th components of $\bar{a}_i$ and $\bar{b}_i$ are a_{iq} and b_{iq} respectively. Show that one can simplify the above expression to the following:*

$$\bar{x}^T(S_1 \odot S_2)\bar{x} = \sum_i \sum_j x_i x_j \left[\sum_k a_{ik} a_{jk} \right] \left[\sum_l b_{il} b_{jl} \right]$$

- *Show that one can simplify the above expression as follows:*

$$\bar{x}^T(S_1 \odot S_2)\bar{x} = \sum_k \sum_l \left[\sum_i x_i a_{ik} b_{il} \right] \left[\sum_j x_j a_{jk} b_{jl} \right]$$

Discuss why this expression is always nonnegative, and therefore the matrix $S_1 \odot S_2$ is positive semidefinite.

Problem 9.7.3 *Show that adding a non-negative value c to each entry of a positive semidefinite matrix does not affect its positive semidefinite property.*

The aforementioned list of properties of positive semidefinite matrices map to properties of positive semidefinite kernels (in closed form as in Table 9.1). First, we define the notion of a (closed-form) positive semidefinite kernel function:

Definition 9.7.1 *A kernel function is positive semidefinite if and only if all possible matrices created by samples of the arguments of that function are positive semidefinite.*

For example, in order to show that the polynomial kernel of Table 9.1 is positive semidefinite, we will have to show that *any* $n \times n$ similarity matrix $P = [p_{ij}]$ created from *arbitrary* $\bar{X}_1 \dots \bar{X}_n \in \mathcal{R}^d$ using the function $p_{ij} = (c + \bar{X}_i \cdot \bar{X}_j)^h$ and $c \geq 0$ is positive semidefinite. The value of n can also be arbitrary, whereas h is a positive integer.

Lemma 9.7.1 (Polynomial Kernel Is Positive Semidefinite) *The $n \times n$ similarity matrix $P = [p_{ij}]$ defined by the polynomial kernel $p_{ij} = (\bar{X}_i \cdot \bar{X}_j + c)^h$ for any $\bar{X}_1 \dots \bar{X}_n \in \mathcal{R}^d$ and $c \geq 0$ is positive semidefinite.*

Proof: Let $S = [s_{ij}]$ be an $n \times n$ matrix in which $s_{ij} = \bar{X}_i \cdot \bar{X}_j$. We already know that the matrix S is positive semidefinite because it is a dot product (Gram) matrix. Let C be an $n \times n$ matrix containing c in each entry. Since $c \geq 0$, it follows that C is positive semidefinite, and the matrix $C + S$ is positive semidefinite as well. The polynomial kernel P can be expressed in the following form:

$$P = (C + S) \odot (C + S) \odot \dots \odot (C + S)$$

From Schur's product theorem, the matrix P is positive semidefinite as well. ■

From Definition 9.7.1, this means that the polynomial kernel is positive semidefinite. One can also show that the Gaussian kernel is positive semidefinite.

Lemma 9.7.2 (Gaussian Kernel Is Positive Semidefinite) *The $n \times n$ similarity matrix $G = [g_{ij}]$ defined by the Gaussian kernel $g_{ij} = \exp(-\|\bar{X}_i - \bar{X}_j\|^2 / (2 \cdot \sigma^2))$ for any $\bar{X}_1 \dots \bar{X}_n \in \mathcal{R}^d$ is positive semidefinite.*

Proof: Let $h_{ij} = \frac{\bar{X}_i}{\sigma} \cdot \frac{\bar{X}_j}{\sigma}$. Then, the matrix $H = [h_{ij}]$ is positive semidefinite, since it is a Gram matrix of dot products. Therefore, the matrix defined by $s_{ij} = \exp(g_{ij})$ is also positive semidefinite, since the (element-wise) exponentiation operation does not affect positive semidefiniteness (see list of properties at the beginning of the section). Define $\delta_i = \exp(-\|\bar{X}_i\|^2 / (2\sigma^2))$. Then, it is easy to show the following:

$$\delta_i s_{ij} \delta_j = \exp(-\|\bar{X}_i - \bar{X}_j\|^2 / (2 \cdot \sigma^2)) = g_{ij}$$

Since the matrix $S = [s_{ij}]$ is positive semidefinite and scaling does not affect positive semidefiniteness, it follows that the matrix $G = [g_{ij}]$ is positive semidefinite as well. ■

Interestingly, the sigmoid kernel is not always positive semi-definite, but works well in practice. One can also create more complicated kernel functions by combining multiple kernels.

Problem 9.7.4 *Use the matrix transformation properties preserving positive semidefiniteness (listed at the beginning of this section) together with Property 9.7.1 to show that (i) The kernel function defined by the sum of two kernel functions is positive semidefinite, (ii) the kernel function defined by the product of two kernel functions is positive semidefinite, and (iii) the kernel function defined by any polynomial function of a kernel function with nonnegative coefficients is positive semidefinite.*

9.8 Summary

Many forms of data are not multidimensional, and examples include discrete sequences and graphs. In such cases, one might have similarities available between objects, but one might not have any multidimensional representation of the data. Eigendecomposition methods from linear algebra help in converting such similarity matrices to multidimensional embeddings. This chapter discusses the use of the similarity matrix in lieu of the multidimensional representation in order to implement machine learning algorithms with similarities rather than multidimensional representations. Similarity-based representations also allow the flattening of nonlinear relationships in the data, so that linear learners become more effective.

9.9 Further Reading

This chapter discusses the linear algebra of similarity matrices and kernel methods. The basics of kernel methods follow from Mercer's theorem [118]. The book by Schölkopf and Smola [118] provides a lot of detail about mathematical properties of kernels and their use in machine learning. The kernel PCA technique is discussed in [112]. The ISOMAP method is introduced in [126]. The kernel outlier detection technique is discussed in [5]. Kernel one-class SVMs are discussed in [113]. The representer theorem is due to Wahba [129].

9.10 Exercises

1. Suppose that you are given a 10×10 *binary* matrix of similarities between objects. The similarities between all pairs of objects for the first four objects is 1, and also

between all pairs of objects for the next six objects is 1. All other similarities are 0. Derive an embedding of each object.

2. Suppose that you have two non-disjoint sets of objects A and B . The set $A \cap B$ is a modestly large sample of objects. You are given all similarities between pairs of objects, one drawn from each of the two sets. Discuss how you can efficiently approximate the entire similarity matrix over the entire set $A \cup B$. It is known that the similarity matrix is symmetric. [Hint: Think of the connections of this setting with matrix factorization. After all, all embeddings are extracted as symmetric matrix factorizations. Here, we are given only a block of the similarity matrix.]
3. Suppose that S_1 and S_2 are $n \times n$ positive semidefinite matrices of ranks k_1 and k_2 , respectively, where $k_2 > k_1$. Show that $S_1 - S_2$ can never be positive semidefinite.
4. Suppose you are given a binary matrix of similarities between objects, in which most entries are 0s. Discuss how you can adapt the logistic matrix factorization approach of Chapter 8 to make it more suitable to symmetric matrix factorization.
5. Suppose that you were given an incomplete matrix of similarities between objects belonging to two sets A and B that are completely disjoint (unlike Exercise 2). Discuss how you can find an embedding for each of the objects in the two sets. Are the embeddings of the objects in set A comparable to those in the set B ?
6. A centered vector is one whose elements sum to 0. Show that for any valid (squared) distance matrix $\Delta = [\delta_{ij}^2]$ defined on a Euclidean space, the following must be true for any *centered* d -dimensional vector $\bar{y}$:

$$\bar{y}^T \Delta \bar{y} \leq 0$$

- (a) Suppose that you are given a symmetric matrix Δ in which all entries along the diagonal are 0s, and it always satisfies $\bar{y}^T \Delta \bar{y} \leq 0$ for all centered $\bar{y}$. Show that all entries of Δ must be nonnegative by using an appropriate choice of vector $\bar{y}$.
 - (b) Discuss why a distance matrix Δ of (squared) Euclidean distances is always indefinite, unless it is a trivial matrix of 0s.
7. You have an $n \times n$ (dot-product) similarity matrix between training points and a $t \times n$ similarity matrix S_t between test and training points. The n -dimensional column vector of class variables is $\bar{y}$. Furthermore, the true $n \times d$ data matrix is D (i.e., $S = DD^T$), but you are not shown this matrix. As discussed in Chapter 4, the d -dimensional coefficient vector $\bar{W}$ of linear regression is given by the following:

$$\bar{W} = (D^T D + \lambda I)^{-1} D^T \bar{y}$$

Here, λ is the regularization parameter. Then, show the following results:

- (a) Let $\bar{p}$ be the t -dimensional vector of predictions for test instances. Show the following using the push-through identity of Problem 1.2.13:

$$\bar{p} = S_t(S + \lambda I)^{-1} \bar{y}$$

- (b) The previous exercise performs differentiation with respect to the weight vector. Show the result of (a) using the representer-based loss function discussed in this chapter, and differentiating with respect to $\bar{\beta}$.

- (c) Take a moment to examine the coefficient vector obtained using the dual approach in Equation 6.14 of Chapter 6 and compare it to one in this exercise. What do you observe?
8. Derive the gradient descent steps for the primal formulation of logistic regression using the similarity matrix S and the representer theorem.
9. A student is given a square and symmetric similarity matrix S that is not positive semidefinite. The student computes the following new matrix:

$$S' = I - S + S^2$$

Is the new similarity matrix S' always positive semidefinite? If it is positive semidefinite, provide a proof. Otherwise, provide a counterexample.

10. A student used three different experimental ways to estimate $n \times n$ similarity matrices S_1 , S_2 , and S_3 among a set of n objects. These similarity matrices were all positive semidefinite. The student then computed the composite similarity matrix S as follows:

$$S = S_1 \odot S_2 + S_2 \odot S_3 + S_3 \odot S_1$$

Is the composite similarity matrix positive semidefinite?

11. Suppose $S(\bar{X}_1, \bar{X}_2) = S(\bar{X}_2, \bar{X}_1)$ is a symmetric similarity function between vectors $\bar{X}_1$ and $\bar{X}_2$, which is not necessarily a valid kernel. Then, is the similarity function $K(\bar{X}_1, \bar{X}_2) = S(\bar{X}_1, \bar{X}_2)^2$ a valid kernel? Either provide a proof or a counter-example.
12. Suppose that S is a positive semidefinite kernel, and a sub-linear element-wise function $f(\cdot)$ is applied to each element of S to create the new matrix $f(S)$. In each case, either show that $f(S)$ is positive semidefinite or provide a counter-example: (i) $f(x)$ is the natural logarithmic function, and S originally contains positive entries, and (ii) $f(x)$ is the non-negative square-root function, and S is originally nonnegative.
13. **Symmetric nonnegative factorization:** Consider a symmetric and nonnegative $n \times n$ matrix S that is factorized as $S \approx UU^T$, where U is a **nonnegative** $n \times k$ matrix for some $k < n$. The errors on the diagonal entries are ignored with the use of the objective function $\|W \odot (S - UU^T)\|^2$. Here, W is an $n \times n$ binary weight matrix that is set to 1 for all entries other than the diagonal ones. Derive a projected gradient-descent update for this box-constrained optimization problem. Discuss why the factor matrix U is more interpretable in the nonnegative case.
14. Show that at least one symmetric factorization $S = UU^T$ exists of a positive semidefinite matrix S , so that U is symmetric as well.
15. Express the loss function of the regularized L_2 -loss SVM (cf. Chapter 5) using the representer theorem in terms of a similarity matrix. Here, we will convert the regularized Newton update of Chapter 5 to a representer update. The Newton update of Chapter 5 (using the same notations as the chapter) is as follows:

$$\bar{W} \leftarrow (D_w^T D_w + \lambda I_d)^{-1} D_w^T \bar{y}$$

Here, the $n \times d$ matrix $D_w = \Delta_w D$ is a partial copy of data matrix D in **feature space**, except that it has zero rows for margin-satisfying rows of D . Δ_w is a binary

diagonal matrix in which the i th diagonal entry is 1 only if the i th training instance of D is margin-violating. How would you compute Δ_w with representer coefficients? Use the push-through identity to show that this update is equivalent to the following with representer coefficients $\bar{\beta}$:

$$D^T \bar{\beta} \Leftarrow D^T \Delta_w (S_w + \lambda I_n)^{-1} \bar{y}$$

Note that one can implicitly implement this update using the following:

$$\bar{\beta} \Leftarrow \Delta_w (S_w + \lambda I_n)^{-1} \bar{y}$$

Here, $S_w = D_w D_w^T$ is a similarity matrix.

16. Consider loss functions of the following form (same notations as text):

$$\text{Minimize } J = \sum_{i=1}^n L(y_i, \bar{W} \cdot \bar{X}_i) + \frac{\lambda}{2} \|\bar{W}\|^2$$

Show that the gradient descent update is as follows:

$$\bar{W} \Leftarrow \bar{W}(1 - \alpha\lambda) - \sum_{i=1}^n \frac{\partial L(y_i, \bar{W} \cdot \bar{X}_i)}{\partial(\bar{W} \cdot \bar{X}_i)} \bar{X}_i^T$$

Now imagine that you only had similarities $S = [s_{ij}]$ available to you. Show that $\bar{W}$ can be updated indirectly by updating its representer coefficients $\bar{\beta}$ as follows:

$$\beta_i \Leftarrow \beta_i(1 - \alpha\lambda) - \alpha \frac{\partial L(y_i, t_i)}{\partial t_i} \quad \forall i \in \{1 \dots n\}$$

Here, we define $t_i = \sum_{p=1}^n s_{ip}$.